

Conservation Corrector — Integration Spec (for SLEEK / PFPL)

Alika M. Parks — for M. Burtscher

October 2026

Reference code: `conserve.h` (public-domain reference; CPU reference + CUDA mapping in comments). This spec is what you need to wire it into SLEEK / PFPL. Scope is the **absolute error bound (ABS)**; the relative-bound variant is noted at the end.

What it does

A lossy codec preserves a pointwise error bound but not integrated invariants (mass, energy, momentum). The corrector restores chosen invariants **exactly**, in closed form, from two stored scalars, touching only the decompressed array — so it is codec-agnostic and drops in after decode. The whole correction is a single affine map $x \leftarrow a \cdot x + b$, so **two scalars (a,b) cover every case** (mass, energy, momentum, mass+energy). Verified on a reference field: mass and energy drift $7.5e-5 \rightarrow 3e-8$ (float32 floor).

Three integration points

1. **At compression — compute and store (a,b)**. Because PFPL (and SLEEK's bound-checked path) already decompresses the data during compression to verify the error bound, you have the reconstructed field $\hat{x}$ in hand at encode time. Compute the correction's two scalars there and store them as side-info in place of the raw invariants:

- mass+energy: `conserve_mass_energy_params( $\hat{x}$ , N, M0, S0, &a, &b)`
- mass (or a momentum component): `conserve_linear_params( $\hat{x}$ , N, L0, w, &a, &b)  $\rightarrow a = 1$`
- energy only: `conserve_energy_params( $\hat{x}$ , N, S0, &a, &b)  $\rightarrow b = 0$`

$M0=\Sigma x$, $S0=\Sigma x^2$, $L0=\Sigma(w \cdot x)$ are taken from the **original** field. Only (a,b) are stored — two doubles per field/block, the only bytes the layer adds. (Fallback for codecs that do *not* decompress at encode: store $(M0,S0)$ instead and let the decoder recover (a,b) with one reduction pass — see point 3.)

2. **Compress at a slightly tightened error bound.** The correction moves each value by a known displacement Δ , so run the codec at a tightened bound $b' = (1-\epsilon) \cdot b$ to keep the *total* error inside the user's requested bound b . This holds for **any** b (uniform tightening — no special-casing of rounding buckets).
 - **mass / momentum (offset):** the displacement is the uniform offset, $|\Delta| = |b| \leq b'$, so a small ϵ suffices and $b' + |b| \leq b$ is checkable at encode.
 - **mass+energy (affine):** the displacement $(a-1)(\hat{x}_i - \text{mean}) + \beta_0$ grows with distance from the mean, so a flat ϵ is **not** sufficient. Use `conserve_mass_energy_maxdisp( $\hat{x}$ ,  $N$ ,  $M0$ ,  $S0$ )` to get $\max_i |\Delta_i|$, then either tighten by it or — since you already have $\hat{x}$ and the original — apply (a,b) , check the corrected field against b , and retighten on the rare miss. Either gives a hard guarantee. Report the small ratio cost of the tightening.

3. **After decompression — correct in place, one pass.** With (a,b) stored:

- `conserve_apply_affine( $x$ ,  $N$ ,  $a$ ,  $b$ )` $\rightarrow x[i] = a \cdot x[i] + b$.

That's a single elementwise pass, **no reduction** on the decode side. (Fallback path, if you stored $(M0,S0)$ instead: `conserve_mass_energy( $x$ ,  $N$ ,  $M0$ ,  $S0$ )` / `conserve_linear( $x$ ,  $N$ ,  $L0$ )`, which do the reduction at decode.)

Modes: standalone CLI vs. API

- **CLI (standalone SLEEK/PFPL):** expose **mass** and **energy** — each is single-field with a scalar target, so the user just names the invariant (like the error bound).
- **API / library: momentum** lives here. Its weight w is the **density field** ($\text{momentum} = \Sigma p_i v_i$), i.e. another field in the dataset — not values the user types. The caller passes the density field pointer as w ; the

encoder uses it to compute the single scalar b , and the **decoder needs no vector at all** (it just applies $(1,b)$). If density is itself corrected (mass), correct it first and use the corrected $\hat{p}$ as w .

The method (why it is exact and always real)

For mass+energy the correction is a **decoupled affine**: an offset sets the mean (fixing mass) and a scale about the mean sets the variance (fixing energy). Scaling about the mean leaves the sum unchanged and the offset leaves the variance unchanged, so the two never interfere — exact and always real (no root-selection, unlike a raw $a \cdot x + b$ match). Collapsing to $a \cdot \hat{x} + b$ with $b = \text{mean0} - a \cdot \text{mean}$ reproduces it exactly in one pass. Full derivation in `conserve.h`.

Taxonomy: linear invariant $\rightarrow$ offset; quadratic invariant $\rightarrow$ scale-about-mean; invariants on separate fields are independent; k invariants on one field $\rightarrow$ a small k -parameter solve (the 2×2 energy+momentum case is written out in `conserve.h`).

GPU mapping (per field/block)

- **Encode:** the reductions $\sum \hat{x}$, $\sum \hat{x}^2$ (fused kernel or `cub::DeviceReduce`, double or Kahan) happen in the decompress you already run for bound-checking; compute (a,b) on the host (2–3 scalars) and store them.
- **Decode:** one elementwise FMA kernel, $x[i] = a \cdot x[i] + b$ — **no device-wide reduction, no host round-trip**. Fuses naturally with SLEEK's own decode sweep.

Cost is $O(N)$ traffic, negligible next to the codec.

What we need from the integration (for the paper)

- Real coded **ratios** (SLEEK/PFPL full pipeline), layer on/off.
- **Throughput** with and without the corrector, on your GPU system(s).
- Invariant drift, layer on/off, confirming the $\sim$ machine-precision restore holds in the GPU path as it does on CPU.

Verification checklist

- ☐ After correction, $|\Sigma x - M0|/|M0|$ and $|\Sigma x^2 - S0|/|S0|$ at the float floor ($\sim 1e-7$ - $1e-8$ in fp32).
- ☐ Max pointwise error stays within the user's bound b (offset: via ϵ ; mass+energy: via `maxdisp/verify`).
- ☐ Single-pass `conserve_apply_affine(a,b)` matches the two-pass reference bit-for-bit (up to fp rounding).
- ☐ Throughput overhead is a small fraction of decode time.